

# Het integreren van Python modules in een .NET rekenmodel

## Een proof-of-concept bij ILVO

Caradoc Roetynck

caradoc.roetynck@student.hogent.be

Promotor: Jan Willem

Co-promotor: Samuel Bosch (ILVO)

Hogeschool Gent, Valentin Vaerwyckweg 1, 9000 Gent

### Samenvatting

Bij ILVO (Instituut voor Landbouw-, Visserij- en Voedingsonderzoek) ontwerpen onderzoekers complexe rekenmodellen die door IT-developers in .NET worden geïmplementeerd. Hierdoor kunnen onderzoekers hun modellen niet altijd zelf aanpassen of snel testen, wat de experimenteerruimte beperkt. Dit onderzoek toont aan dat door de integratie van Python met de bestaande C#-code, onderzoekers de autonomie krijgen om modellen direct te valideren en aan te passen, zonder de stabiliteit van de hoofdapplicatie te schaden. Naast de Python gaat dit onderzoek ook diep in Clean Code en Documentatie Drift, aangezien dit belangrijke zaken zijn om code te verduidelijken.

Keuzerichting: Software Development

Sleutelwoorden: Docker, C#, Jupyter-Notebooks, Razor

Broncode:

## 1. Introductie

Bij ILVO worden complexe rekenmodellen gebruikt om milieu-impact en emissies in de landbouw te berekenen. Deze modellen ontstaan vanuit wetenschappelijke kennis, maar worden in de praktijk omgezet naar software in een .NET/C#-omgeving. Daardoor ontstaat een duidelijke kloof tussen de wetenschappelijke logica en de technische implementatie.

### De uitdaging

Hoewel de verdeling tussen onderzoekers en ontwikkelaars logisch is, leidt ze in de praktijk tot twee problemen:

- Beperkte zichtbaarheid: de berekeningslogica is verspreid over broncode, documentatie en overleg, waardoor onderzoekers de stappen moeilijk kunnen verifiëren.
- Afhankelijkheid: voor elke wijziging of test van een formule is technische hulp nodig, wat de experimenteerruimte beperkt en de snelheid van onderzoek vertraagt.

Het probleem is niet alleen technisch, maar ook organisatorisch: documentatie raakt snel achter op de implementatie. Daardoor ontstaat *documentation drift*, een groeiende kloof tussen wat de documentatie zegt en wat de software werkelijk doet.

### Onderzoeksvraag

Hoe kunnen we de kloof tussen IT-logica en wetenschappelijke analyse verkleinen, zodat onderzoekers zonder diepgaande programmeerkennis inzicht krijgen in hun modellen en er zelf mee kunnen experimenteren?

## 2. Methodologie & Technologie

Het onderzoek is opgebouwd in een iteratief proces: gesprekken met onderzoekers en IT-medewerkers, literatuuronderzoek en een praktische proof-of-concept. De verzamelde inzichten worden steeds teruggekoppeld om de oplossing te verfijnen.

### Gekozen aanpak:

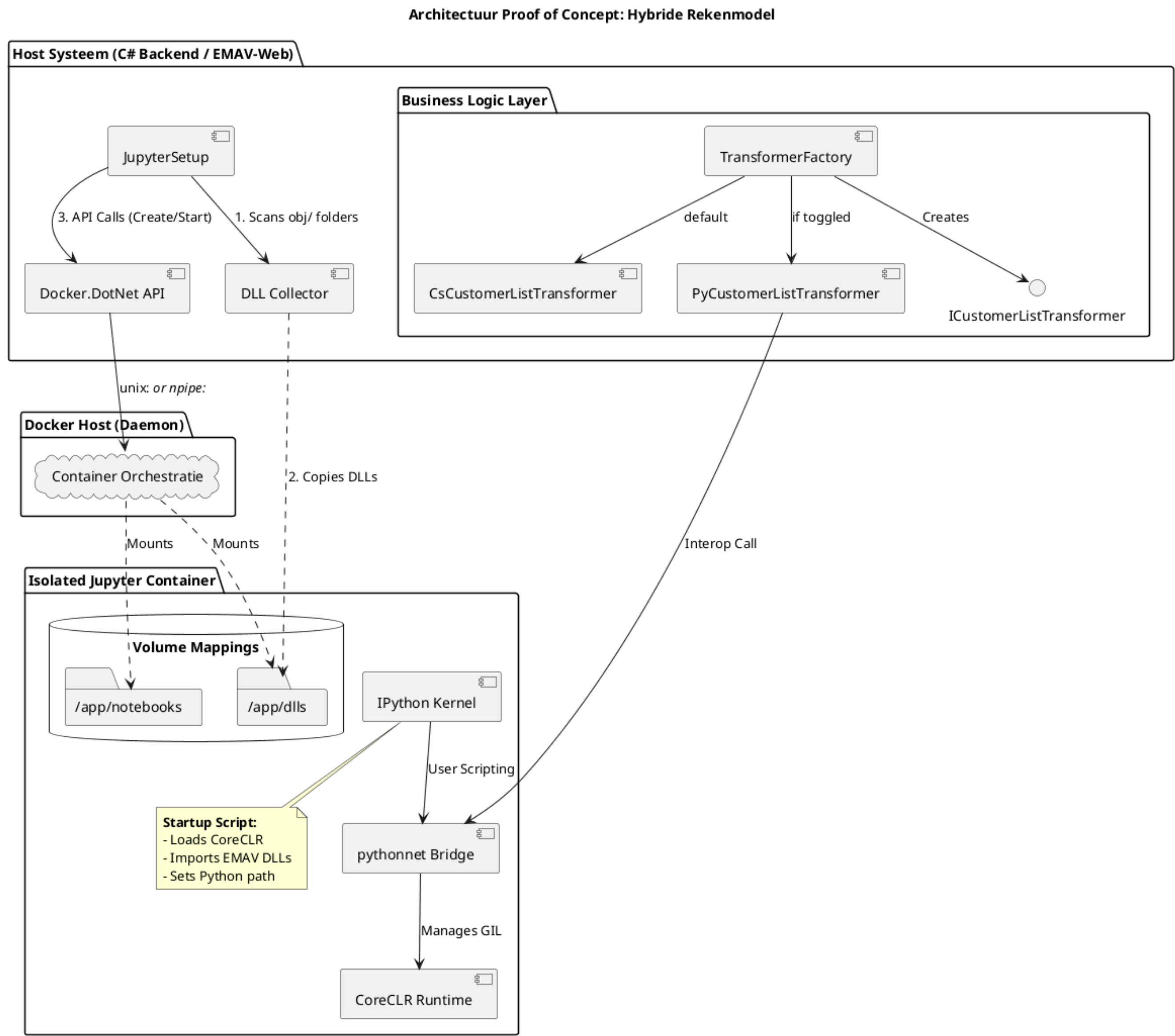
- Probleemanalyse: gesprekken met onderzoekers en IT-medewerkers om de huidige werkwijze, knelpunten en behoeften in kaart te brengen.
  - Literatuur- en technologieverkenning: onderzoek naar Python.NET, Jupyter, literate programming, containers en documentatie-praktijken die de kloof tussen model en code kunnen verkleinen.
  - Ontwerp van een hybride oplossing: een proof-of-concept waarbij C#-logica via Python beschikbaar wordt gemaakt in een veilige, interactieve analyseomgeving.
  - Iteratieve evaluatie: feedback uit de praktijk gebruiken om de oplossing te verfijnen en te toetsen op bruikbaarheid, veiligheid en wetenschappelijke validatie.
- Deze aanpak sluit aan bij de kern van het onderzoek: niet alleen een technische integratie, maar ook een methodiek om documentatie, experimenteren en samenwerking tussen onderzoekers en ontwikkelaars beter te laten samenwerken.

## 3. Proof-of-Concept: Hybride Architectuur

De proof-of-concept laat zien hoe een robuuste .NET-backend als bron van waarheid kan dienen, terwijl onderzoekers in een vertrouwde Python-omgeving met de berekeningen kunnen werken. De architectuur is ontworpen om de samenwerking tussen wetenschappelijke analyse en softwareontwikkeling te verbeteren zonder de stabiliteit van de hoofdapplicatie te ondermijnen.

### Kernaspecten van de realisatie:

- Geautomatiseerde orchestration: de C#-backend beheert via Docker de levenscyclus van de analyse-omgevingen, zodat elke onderzoeker een identieke en geïsoleerde setup krijgt.
- Single source of truth: de domeinlogica blijft in de C#-codebase, terwijl onderzoekers deze logica via Python kunnen gebruiken en testen.
- Naadloze interop: dankzij Python.NET kunnen C#-types en methoden direct worden aangeroepen vanuit Python, zonder dat de onderzoeker zelf diep in C# hoeft te werken.
- Interactieve validatie: notebooks dienen als werkruimte waarin resultaten, parameters en scenario's kunnen worden vergeleken in een veilige omgeving.



Figuur 1: Architecturaal overzicht van de PoC met C#-orchestratie en hybride Python-integratie.

## 4. Schakelbare Implementatie

Het kernidee is dat onderzoekers op een gecontroleerde manier kunnen wisselen tussen de officiële implementatie en een experimentele alternatieve variant, zonder de productieomgeving direct te verstoren.

### Het mechanisme:

- Dynamische selectie: op runtime kan een rekenmodule kiezen voor de standaard C#-variant of een Python-variant.
- Granulaire controle: per berekeningsstap kan een alternatief scenario worden getest zonder het hele systeem te beïnvloeden.
- Veiligheid: tijdelijke data- en databasekopieën, gecombineerd met Docker-isolatie, voorkomen dat experimentele berekeningen de productiedata vervuilen.
- Ondersteuning van onderzoek: de switch maakt het mogelijk om snelle experimenten uit te voeren en resultaten te vergelijken met de vaste productie-implementatie.

## 5. Conclusies & Toekomst

De proof-of-concept toont aan dat een hybride aanpak de autonomie van onderzoekers vergroot zonder het vertrouwen in de bestaande .NET-backend te ondermijnen. De combinatie van Python, Jupyter en veilige isolatie maakt het mogelijk om complexe berekeningen beter te begrijpen, te testen en te valideren.

### Toekomstig onderzoek:

- Structureel inbedden van de switch-logica in de verschillende EMAV-rekenmodules.
- Uitbreiden van notebook-validatietools voor automatische vergelijking tussen C#- en Python-resultaten.
- Verdiepen van de samenwerking met onderzoekers om de experimenteerruimte te optimaliseren.
- Verder uitwerken van de PoC en de documentatiepraktijk om drift te beperken.